LearnRAG

Research Assistant — Hybrid Retrieval & Reranking RAG Pipeline

This document describes the design, components, and data flow of the LearnRAG research assistant: a Retrieval-Augmented Generation (RAG) system that ingests a corpus of ~200 arXiv research papers on LLM evaluation, and answers questions grounded in that corpus using hybrid (dense + sparse) retrieval, cross-encoder reranking, and Groq-hosted LLM generation.

This project evolved from an earlier single-PDF RAG prototype (LearnRAG) into a multi-document research assistant, built as an extension on a dedicated branch.

1. Project Overview

RAG (Retrieval-Augmented Generation) improves LLM answers by first retrieving relevant information from an external knowledge source, then providing that information to the model as context before it generates a response. This grounds output in real source material rather than relying purely on the model's pretrained knowledge.

This version of LearnRAG extends the original single-document prototype into a genuine research assistant: it ingests roughly 200 academic papers on a focused topic (LLM evaluation), and combines two complementary retrieval strategies — dense semantic search and sparse keyword search — followed by a cross-encoder reranking stage, before handing context to the LLM. This two-stage retrieve-then-rerank design substantially improves answer precision over dense-only retrieval, particularly for queries involving exact terminology (benchmark names, metrics, acronyms) that pure semantic search can under-match.

2. Pipeline Architecture

Stage	Module	Description
0. Acquire	download_papers.py	Downloads ~200 arXiv papers on a chosen research topic via the arXiv API.
1. Load	document_loader.py	Parses all PDFs (PyMuPDF); caches parsed documents to disk to avoid re-parsing on every run.
2. Chunk	chunker.py	Splits into token-sized chunks; filters near-empty and reference-list fragments.
3. Embed	embedder.py	Converts chunks into vectors using a local HuggingFace embedding model (free).
4. Store	vectordb.py	Batched embedding + storage into a persistent Chroma vector database.
5. Retrieve	retriever.py	Hybrid search: dense (Chroma) + sparse (BM25), fused via Reciprocal Rank Fusion.
6. Rerank	reranker.py	Cross-encoder rescoring of candidates for final precision (top-k narrowing).
7. Generate	generator.py	Reranked context + query -> Groq LLM -> grounded, context-only answer.

Data Flow Diagram

```
arXiv Papers → document_loader.py → chunker.py → embedder.py → vectordb.py (Chroma) → retriever.py (hybrid RRF) → reranker.py (cross-encoder) → generator.py (Groq LLM) → Grounded Answer
```

Why Two Separate Stages: Retrieval vs. Reranking

Retrieval (dense + sparse + RRF) is optimized for recall: quickly narrowing a large corpus (thousands of chunks) down to a reasonable candidate set. A cross-encoder reranker is substantially more accurate at judging true relevance, because it scores the query and each candidate chunk jointly as a single input, rather than comparing independently-computed vectors or keyword scores. It is too slow to run over an entire corpus, so the design retrieves broadly first, then reranks narrowly before generation.

3. Module Reference

3.1 download_papers.py

Uses the arxiv Python package to search and download a focused paper set (e.g. ~200 papers matching “LLM evaluation” within the cs.CL category). Downloads each PDF via requests using the result's pdf_url field, since newer versions of the arxiv package removed the built-in download_pdf() convenience method. Already-downloaded files are skipped so the script is safe to re-run.

3.2 document_loader.py

- **pdf_loader()** — loads a single PDF via PyMuPDFLoader, chosen over pypdf for faster, more robust extraction on multi-column academic layouts.
- **load_pdf_folder()** — loads every PDF in a folder recursively, catching and logging failures (corrupt or scanned/image-only files) instead of crashing the batch.
- **get_documents()** — wraps folder loading with a pickle-based disk cache, so repeated runs during development don't re-parse ~200 PDFs each time.

3.3 chunker.py

Splits documents using RecursiveCharacterTextSplitter, measured by token count (via tiktoken) rather than raw characters for more predictable prompt sizing downstream. Two filters are applied after splitting:

- Near-empty chunks (stray headers, isolated page numbers) are dropped by minimum length.
- Likely bibliography/reference-list fragments are dropped via a heuristic detecting dense bracketed citation numbers or repeated proper-name-comma sequences characteristic of author lists.

Semantic chunking (embedding-based sentence-boundary splitting) was evaluated during early development but replaced with recursive splitting at this corpus scale, since semantic chunking requires an embedding call per sentence boundary and becomes prohibitively slow across ~200 papers.

3.4 embedder.py

Loads a local HuggingFace embedding model (sentence-transformers/all-MiniLM-L6-v2) via HuggingFaceEmbeddings. Runs entirely on-device, producing 384-dimension dense vectors with no API cost. Setting an HF_TOKEN environment variable avoids Hugging Face Hub rate-limit warnings on initial model download.

3.5 vectordb.py

- **build_vector_store()** — embeds chunks and writes them to a persistent Chroma collection in batches, avoiding one large blocking call across ~5,500+ chunks.
- **load_vector_store()** — reloads an existing store from disk without re-embedding.
- **similarity_search()** — plain dense similarity search, retained as a baseline for comparing against hybrid search results.

3.6 retriever.py — Hybrid Search

Combines dense and sparse retrieval into a single ranked candidate set:

- **build_bm25_retriever()** — a sparse keyword retriever (BM25) built directly from chunk text; no embedding model or vector database required.
- **build_hybrid_retriever()** — combines the dense Chroma retriever and the BM25 retriever via LangChain's EnsembleRetriever, which performs weighted Reciprocal Rank Fusion (RRF).
- **hybrid_search()** — runs a query and returns the fused candidate list.

RRF Scoring Formula

For each retriever's ranked result list, every document accumulates a score of weight / (rank + c), where rank is its position in that retriever's list (starting at 1) and c is a smoothing constant (default 60). Scores from multiple retrievers are summed per document (by content), so a chunk found by both dense and sparse search accumulates contributions from each, then the combined list is sorted by total score, descending.

```
rrf_score[doc] += weight / (rank + c)
```

3.7 reranker.py — Cross-Encoder Reranking

Takes the candidate set produced by retriever.py and re-scores each (query, chunk) pair using a cross-encoder model (cross-encoder/ms-marco-MiniLM-L-6-v2), which reads the query and candidate together as one input rather than comparing separately-computed representations. This is more accurate than RRF-fused ranking alone but too computationally expensive to run over an entire corpus, so it operates only on the retriever's candidate set (e.g. narrowing ~15–20 candidates to the top 3–5).

3.8 generator.py

Wires the full pipeline together using LangChain Expression Language (LCEL):

- Accepts a user question.
- Retrieves candidates via hybrid_search(), then narrows via rerank().
- Joins the final top chunks into a single context string.
- Formats context and question into a prompt instructing the model to answer only from the given context, and to respond “I don't know” if the context is insufficient.
- Sends the prompt to a Groq-hosted LLM (openai/gpt-oss-120b) and parses the text response.

4. Technology Stack

Component	Tool / Library
Paper Acquisition	arxiv package (arXiv API)
PDF Loading	PyMuPDFLoader (langchain-community)
Text Splitting	RecursiveCharacterTextSplitter (token-based, via tiktoken)
Embeddings	HuggingFace sentence-transformers/all-MiniLM-L6-v2 (local, free)
Vector Database	ChromaDB (batched writes)
Sparse Retrieval	BM25 (rank_bm25, via BM25Retriever)
Hybrid Fusion	EnsembleRetriever (Reciprocal Rank Fusion)
Reranking	cross-encoder/ms-marco-MiniLM-L-6-v2 (local, free)
LLM Inference	Groq (openai/gpt-oss-120b)
Orchestration	LangChain Expression Language (LCEL)

5. Setup Instructions

Install dependencies:

```
pip install -r requirements.txt
```

Create a .env file in the project root with:

```
GROQ_API_KEY=your_groq_api_key_here
HF_TOKEN=your_huggingface_token_here
```

Download the research paper corpus:

```
python src/learnrag/download_papers.py
```

Run the full pipeline:

```
python src/learnrag/generator.py
```

6. Known Limitations

- Table and figure-caption content can extract as jumbled text fragments, since PDF parsers do not understand table structure; this occasionally surfaces as a noisy retrieval result.
- A small fraction of downloaded papers (roughly 3 of 200) may fail to parse (corrupted download, or scanned/image-only PDFs); these are logged and skipped rather than crashing the pipeline.
- Groq periodically deprecates model IDs (e.g. llama-3.3-70b-versatile was replaced by openai/gpt-oss-120b); check the Groq models documentation if a model_not_found error occurs.

7. Possible Extensions

- **Docker Deployment** — containerize the app; Chroma with a mounted volume suffices for single-host deployment. A managed vector database (e.g. Pinecone) becomes worth considering only for serverless or multi-replica deployments where local disk does not persist or is not shared across instances.
- **Evaluation Harness** — build a test set of question/answer pairs to measure retrieval precision/recall and answer faithfulness.
- **Source-Aware Citations** — surface which paper(s) each answer draws from directly in the response; per-chunk source metadata is already tracked throughout the pipeline.
- **UI Layer** — wrap the pipeline in a Streamlit or Gradio interface for interactive use.